

Добавление элементов

Мы научились создавать множества, элементы которых известны на этапе создания. Следующий шаг – научиться добавлять элементы в уже существующие множества.

Метод `add()`

Для добавления нового элемента в множество используется метод `add()`.

Следующий программный код:

```
numbers = {1, 1, 2, 3, 5, 8, 3} # создаем множество
numbers.add(21) # добавляем число 21 в множество
numbers.add(34) # добавляем число 34 в множество
print(numbers)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 3, 34, 5, 8, 21}
```

Не забывайте, что порядок элементов при выводе множества абсолютно произвольный.

Обратите внимание, для использования метода `add()` требуется предварительно созданное множество, при этом оно может быть пустым.

Следующий программный код:

```
numbers = set() # создаем пустое множество
numbers.add(1)
numbers.add(2)
numbers.add(3)
numbers.add(1)
print(numbers)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 3}
```

Если требуется внести несколько значений в множество, то можно воспользоваться циклом `for`.

Следующий программный код:

```
numbers = set() # создаем пустое множество
for i in range(10):
    numbers.add(i*i + 1)
print(numbers)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 65, 5, 37, 10, 17, 50, 82, 26}
```

Удаление элемента

Для удаления элементов из множества используются методы:

- `remove()`;
- `discard()`;
- `pop()`.

Метод `remove()`

Метод `remove()` — удаляет элемент из множества с генерацией исключения (ошибки) в случае, если такого элемента нет.

Следующий программный код:

```
numbers = {1, 2, 3, 4, 5}
numbers.remove(3)
print(numbers)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 4, 5}
```

Следующий программный код:

```
numbers = {1, 2, 3, 4, 5}
```

```
numbers.remove(10)
```

```
print(numbers)
```

приводит к возникновению ошибки `KeyError`, так как элемент

10

10 отсутствует в множестве.

Метод `discard()`

Метод `discard()` — удаляет элемент из множества без генерации исключения (ошибки), если элемент отсутствует.

Следующий программный код:

```
numbers = {1, 2, 3, 4, 5}
```

```
numbers.discard(3)
```

```
print(numbers)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 4, 5}
```

Следующий программный код:

```
numbers = {1, 2, 3, 4, 5}
```

```
numbers.discard(10)
```

```
print(numbers)
```

не приводит к возникновению ошибки и выводит (порядок элементов может отличаться):

```
{1, 2, 3, 4, 5}
```

Метод `pop()`

Метод `pop()` — удаляет и возвращает случайный элемент из множества с генерацией исключения (ошибки) при попытке удаления из пустого множества.

Рассмотрим программный код:

```
numbers = {1, 2, 3, 4, 5}

print('до удаления:', numbers)

num = numbers.pop()          # удаляет случайный элемент
                               множества, возвращая его

print('удалённый элемент:', num)

print('после удаления:', numbers)
```

Результат работы такого кода случаен, например, такой код может вывести:

```
до удаления: {1, 2, 3, 4, 5}

удалённый элемент: 1

после удаления: {2, 3, 4, 5}
```

Метод `pop()` можно воспринимать как неконтролируемый способ удаления элементов по одному из множества.

Метод `clear()`

Метод `clear()` удаляет все элементы из множества.

Следующий программный код:

```
numbers = {1, 2, 3, 4, 5}

numbers.clear()

print(numbers)
```

выведет:

```
set()
```

В результате получили пустое множество.

Обратите внимание на то, что пустое множество выводится как `set()`, а не как `{}`. С помощью `{}` выводится пустой словарь.

Примечания

Примечание 1. Если мы не изменяли множество, порядок обхода элементов при помощи цикла `for` не изменится.

Примечание 2. После изменения множества (методы `add()`, `remove()`, и т.д.) порядок элементов может измениться произвольным образом.

Операции над множествами

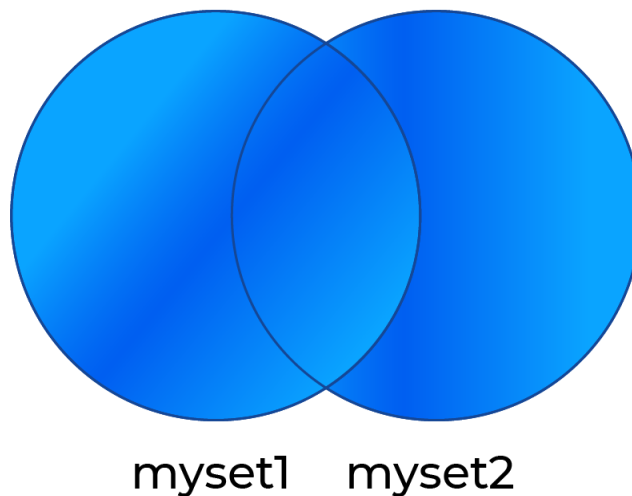
Основные операции над множествами:

- объединение множеств;
- пересечение множеств;
- разность множеств;
- симметрическая разность множеств.

Для каждой операции есть метод и оператор.

Объединение множеств: метод `union()`

Объединение множеств это множество, состоящее из элементов, принадлежащих хотя бы одному из объединяемых множеств. Для этой операции существует метод `union()`.



Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset3 = myset1.union(myset2)
print(myset3)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

Обратите внимание, метод `union()` **возвращает новое множество** в которое входят все элементы множеств `myset1` и `myset2`. Для изменения текущего множества используется метод `update()`.

Для объединения двух множеств можно также использовать оператор `|`.

Результат выполнения приведенного ниже кода:

```
myset1 = {1, 2, 3, 4, 5}
```

```
myset2 = {3, 4, 6, 7, 8}
```

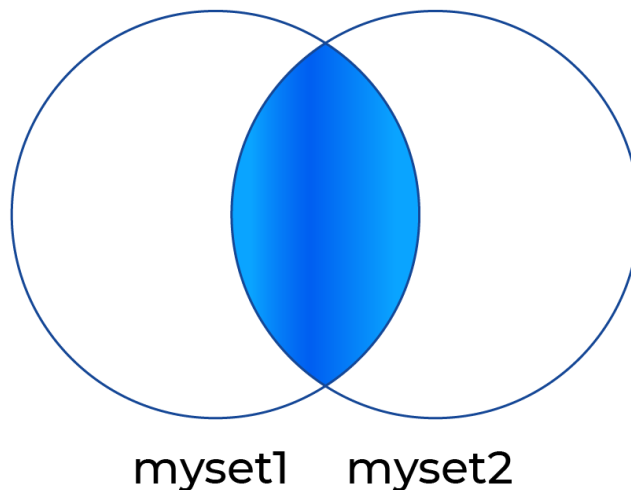
```
myset3 = myset1 | myset2
```

```
print(myset3)
```

аналогичен предыдущему.

Пересечение множеств: метод `intersection()`

Пересечение множеств это множество, состоящее из элементов, принадлежащих одновременно каждому из пересекающихся множеств. Для этой операции существует метод `intersection()`.



Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5}
```

```
myset2 = {3, 4, 6, 7, 8}
```

```
myset3 = myset1.intersection(myset2)

print(myset3)
```

выводит (порядок элементов может отличаться):

```
{3, 4}
```

Обратите внимание, метод `intersection()` возвращает новое множество в которое входят общие элементы множеств `myset1` и `myset2`. Для изменения текущего множества используется метод `intersection_update()`.

Для пересечения двух множеств можно также использовать оператор `&`.

Результат выполнения приведенного ниже кода:

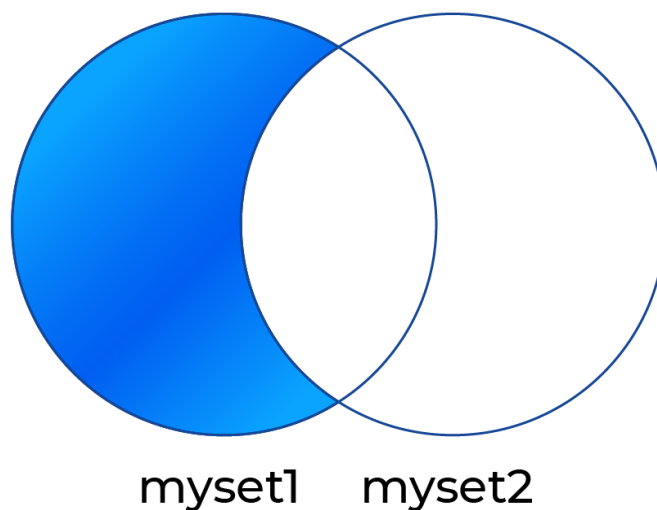
```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset3 = myset1 & myset2

print(myset3)
```

аналогичен предыдущему.

Разность множеств: метод `difference()`

Разность множеств это множество, в которое входят все элементы первого множества, не входящие во второе множество. Для этой операции существует метод `difference()`.



Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset3 = myset1.difference(myset2)
print(myset3)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 5}
```

Для разности двух множеств можно также использовать оператор `-`.

Результат выполнения приведенного ниже кода:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset3 = myset1 - myset2
print(myset3)
```

аналогичен предыдущему.

Обратите внимание: для операции разности множеств важен порядок, в котором указаны множества. Если поменять местами `myset1` и `myset2`, нас ожидает совсем другой результат: элементы входящие в множество `myset2` и которых нет в множестве `myset1`.

Приведенный ниже код:

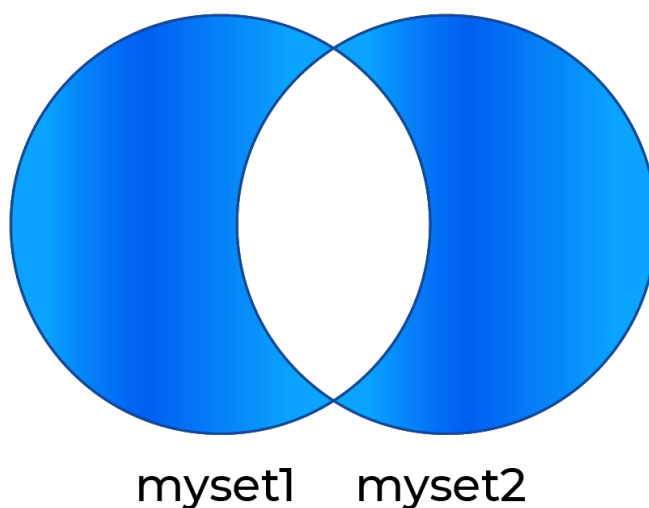
```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset3 = myset2.difference(myset1)
print(myset3)
```

выводит (порядок элементов может отличаться):

```
{8, 6, 7}
```

Симметрическая разность: метод `symmetric_difference()`

Симметрическая разность множеств это множество, включающее все элементы исходных множеств, не принадлежащие одновременно обоим исходным множествам. Для этой операции существует метод `symmetric_difference()`.



Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset3 = myset1.symmetric_difference(myset2)
print(myset3)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 5, 6, 7, 8}
```

Для симметрической разности двух множеств можно также использовать оператор `^`.

Результат выполнения приведенного ниже кода:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
```

```
myset3 = myset1 ^ myset2  
  
print(myset3)
```

аналогичен предыдущему.

Обратите внимание: для операции симметрической разности порядок множеств не важен, на то она и симметрическая: `myset1 ^ myset2 == myset2 ^ myset1`.

Методы множеств, изменяющие текущие множества

Методы `union()`, `intersection()`, `difference()`, `symmetric_difference()` не изменяют исходные множества, а возвращают новые. Часто на практике нужно изменять исходные множества. Для таких целей используются парные методы `update()`, `intersection_update()`, `difference_update()`, `symmetric_difference_update()`.

Метод `update()`

Метод `update()` изменяет исходное множество **по объединению**.

Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5}  
  
myset2 = {3, 4, 6, 7, 8}  
  
myset1.update(myset2)      # изменяем множество myset1  
  
print(myset1)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

Аналогичный результат получается, если использовать оператор `|=`:

```
myset1 = {1, 2, 3, 4, 5}  
  
myset2 = {3, 4, 6, 7, 8}  
  
myset1 |= myset2  
  
print(myset1)
```

Метод intersection_update()

Метод `intersection_update()` изменяет исходное множество **по пересечению**.

Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}

myset1.intersection_update(myset2)    # изменяем множество
myset1

print(myset1)
```

выводит (порядок элементов может отличаться):

```
{3, 4}
```

Аналогичный результат получается, если использовать оператор `&=`:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}

myset1 &= myset2

print(myset1)
```

Метод difference_update()

Метод `difference_update()` изменяет исходное множество **по разности**.

Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}

myset1.difference_update(myset2)    # изменяем множество myset1

print(myset1)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 5}
```

Аналогичный результат получается, если использовать оператор `-=`:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset1 -= myset2
print(myset1)
```

Метод `symmetric_difference_update()`

Метод `symmetric_difference_update()` изменяет исходное множество **по симметрической разности**.

Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset1.symmetric_difference_update(myset2)    # изменяем
множество myset1
print(myset1)
```

выводит (порядок элементов может отличаться):

```
{1, 2, 5, 6, 7, 8}
```

Аналогичный результат получается, если использовать оператор `^=`:

```
myset1 = {1, 2, 3, 4, 5}
myset2 = {3, 4, 6, 7, 8}
myset1 ^= myset2
print(myset1)
```

Примечания

Примечание 1. Все основные операции над множествами выполняются двумя способами: при помощи метода или соответствующего ему оператора. Различие в том, что метод может принимать в качестве аргумента не только множество (тип данных `set`), но и любой итерируемый объект (список, строку, кортеж).

Приведенный ниже код:

```
mylist = [2021, 2020, 2019, 2018, 2017, 2016]
```

```
mytuple = (2021, 2020, 2016)
```

```
mystr = 'abcd'
```

```
myset = {2009, 2010, 2016}
```

```
print(myset.union(mystr))          # объединяем со строкой
```

```
print(myset.intersection(mylist)) # пересекаем со списком
```

```
print(myset.difference(mytuple))  # находим разность с  
кортежем
```

выводит (порядок элементов может отличаться):

```
{2016, 'c', 'b', 'a', 'd', 2009, 2010}
```

```
{2016}
```

```
{2009, 2010}
```

Приведенный ниже код:

```
mylist = [2021, 2020, 2019, 2018, 2017, 2016]
```

```
mytuple = (2021, 2020, 2016)
```

```
mystr = 'abcd'
```

```
myset = {2009, 2010, 2016}
```

```
print(myset | mystr)
```

```
print(myset & mylist)
```

```
print(myset - mytuple)
```

приводит к возникновению ошибок:

```
TypeError: unsupported operand type(s) for |: 'set' and 'str'
```

```
TypeError: unsupported operand type(s) for &: 'set' and 'list'
```

```
TypeError: unsupported operand type(s) for -: 'set' and 'tuple'
```

Примечание 2. Некоторые методы (`union()`, `intersection()`, `difference()`) и операторы (`|`, `&`, `-`, `^`) позволяют совершать операции над несколькими множествами сразу.

Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5, 6}
```

```
myset2 = {2, 3, 4, 5}
```

```
myset3 = {5, 6, 7, 8}
```

```
union1 = myset1.union(myset2, myset3)
```

```
union2 = myset1 | myset2 | myset3
```

```
difference1 = myset1.difference(myset2, myset3)
```

```
difference2 = myset1 - myset2 - myset3      # порядок  
выполнения слева-направо
```

```
print(union1 == union2)
```

```
print(difference1 == difference2)
```

ВЫВОДИТ:

```
True
```

```
True
```

Примечание 3. Оператор `^` симметрической разности позволяет использовать несколько множеств, а метод `symmetric_difference()` – нет.

Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5, 6}
```

```
myset2 = {2, 3, 4, 7}
```

```
myset3 = {6, 20, 30}
```

```
symdifference = myset1 ^ myset2 ^ myset3 # порядок выполнения  
слева-направо
```

```
print(symdifference)
```

выводит (порядок элементов может отличаться):

```
{1, 5, 7, 20, 30}
```

Приведенный ниже код:

```
myset1 = {1, 2, 3, 4, 5, 6}
```

```
myset2 = {2, 3, 4, 7}
```

```
myset3 = {6, 20, 30}
```

```
symdifference = myset1.symmetric_difference(myset2, myset3)
```

```
print(symdifference)
```

приводит к ошибке:

```
TypeError: symmetric_difference() takes exactly one argument (2  
given)
```

Примечание 4. Таблица соответствия методов и операторов над множествами.

<pre>A B</pre> <pre>A.union(B)</pre>	Возвращает множество, являющееся объединением множеств <code>A</code> и <code>B</code>
---	--

$A \mid= B$ <code>A.update(B)</code>	Добавляет в множество <code>A</code> все элементы из множества <code>B</code>
$A \& B$ <code>A.intersection(B)</code>	Возвращает множество, являющееся пересечением множеств <code>A</code> и <code>B</code>
$A \&= B$ <code>A.intersection_update(B)</code>	Оставляет в множестве <code>A</code> только те элементы, которые есть в множестве <code>B</code>
$A - B$ <code>A.difference(B)</code>	Возвращает разность множеств <code>A</code> и <code>B</code>
$A -= B$ <code>A.difference_update(B)</code>	Удаляет из множества <code>A</code> все элементы, входящие в <code>B</code>
$A \wedge B$ <code>A.symmetric_difference(B)</code>	Возвращает симметрическую разность множеств <code>A</code> и <code>B</code>
$A \wedge= B$ <code>A.symmetric_difference_update(B)</code>	Записывает в <code>A</code> симметрическую разность множеств <code>A</code> и <code>B</code>

Примечание 5. Приоритет операторов в порядке убывания (верхние операторы имеют более высокий приоритет, чем нижние) имеет вид:

Оператор	Описание
<code>-</code>	разность

$\&$	пересечение
$\wedge$	симметрическая разность
$\cup$	объединение

Подмножества и надмножества

Напомним, что множество `set1` является **подмножеством** множества `set2`, если все элементы первого входят во второе. При этом множество `set2` – **надмножество** множества `set1`.

Любое множество – подмножество самого себя, про такое подмножество говорят "**нестрогое подмножество**".

Метод `issubset()`

Для определения, является ли одно из множеств подмножеством другого, используется метод `issubset()`. Данный метод возвращает значение `True`, если одно множество является подмножеством другого, и `False`, если не является.

Приведенный ниже код:

```
set1 = {2, 3}

set2 = {1, 2, 3, 4, 5, 6}

print(set1.issubset(set2))
```

ВЫВОДИТ:

```
True
```

В этом примере `set2` содержит все элементы `set1`. Это означает, что `set1` – подмножество `set2`. Это также означает, что `set2` – надмножество `set1`.

Для определения, является ли одно из множеств подмножеством другого, также применяются операторы `<=` (нестрогое подмножество) и `<` (строгое подмножество).

Приведенный ниже код:

```
set1 = {2, 3}

set2 = {1, 2, 3, 4, 5, 6}

print(set1 <= set2)
```

аналогичен предыдущему.

Метод `issuperset()`

Для определения, является ли одно из множеств надмножеством другого, используется метод `issuperset()`. Данный метод возвращает значение `True`, если одно множество является надмножеством другого, в противном случае он возвращает `False`.

Приведенный ниже код:

```
set1 = {'a', 'b', 'c', 'd', 'e'}  
set2 = {'c', 'e'}
```

```
print(set1.issuperset(set2))
```

ВЫВОДИТ:

```
True
```

В этом примере `set1` содержит все элементы `set2`. Это означает, что `set1` – надмножество `set2`. Это также означает, что `set2` – подмножество `set1`.

Для определения, является ли одно из множеств надмножеством другого, также применяются операторы `>=` (нестрогое надмножество) и `>` (строгое надмножество).

Приведенный ниже код:

```
set1 = {'a', 'b', 'c', 'd', 'e'}  
set2 = {'c', 'e'}
```

```
print(set1 >= set2)
```

аналогичен предыдущему.

Метод isdisjoint()

Для определения отсутствия общих элементов в множествах используется метод `isdisjoint()`. Данный метод возвращает значение `True`, если множества не имеют общих элементов, и `False`, когда множества имеют общие элементы.

Приведенный ниже код:

```
set1 = {1, 2, 3, 4, 5}
```

```
set2 = {5, 6, 7}
```

```
set3 = {7, 8, 9}
```

```
print(set1.isdisjoint(set2))
```

```
print(set1.isdisjoint(set3))
```

```
print(set2.isdisjoint(set3))
```

ВЫВОДИТ:

```
False
```

```
True
```

```
False
```

Примечания

Примечание 1. Методы `issuperset()`, `issubset()`, `isdisjoint()` могут принимать в качестве аргумента не только множество (тип данных `set`), но и любой итерируемый объект (список, строку, кортеж...). Сами же эти методы могут применяться только ко множеству (тип данных `set`) или замороженному множеству (тип данных `frozenset`).

Примечание 2. Операторы `>`, `<`, `>=`, `<=` требуют наличия в качестве операндов множества.

Примечание 3. Таблица соответствия методов и операторов над множествами.

<pre>set1 <= set2 set1.issubset(set2)</pre>	Возвращает <code>True</code> , если <code>set1</code> является подмножеством <code>set2</code>
<pre>set1 >= set2 set1.issuperset(set2)</pre>	Возвращает <code>True</code> , если <code>set1</code> является надмножеством <code>set2</code>
<pre>set1 < set2</pre>	Эквивалентно <code>set1 <= set2 and set1 != set2</code> (строгое подмножество)
<pre>set1 > set2</pre>	Эквивалентно <code>set1 >= set2 and set1 != set2</code> (строгое надмножество)